

Service Desk — Full Software Documentation

Version: 1.0 | Last Updated: February 2025 | Document Type: Complete Software Documentation

Table of Contents

- [1. Executive Summary](#)
 - [2. System Architecture](#)
 - [3. Technology Stack](#)
 - [4. Feature Suite — In-Depth](#)
 - [5. Processes & Workflows](#)
 - [6. API Reference](#)
 - [7. Database Schema Summary](#)
 - [8. Frontend Structure & Routes](#)
 - [9. Security & Access Control](#)
 - [10. Setup & Deployment](#)
 - [11. Related Documentation](#)
-

1. Executive Summary

1.1 Purpose

The **Service Desk** (Vodafone Service Desk) is an **enterprise IT support management platform** for:

- Ticket management** — Full lifecycle from creation to closure
- Approval workflows** — Multi-level approval for requests requiring authorization
- SLA tracking** — Response and resolution time monitoring with breach detection
- Team collaboration** — Assignment to teams and individuals, multi-team support
- Asset management** — IT asset inventory and ticket linking
- Knowledge base** — Self-service articles and AI-powered FAQ chat
- Reporting & analytics** — Tickets, agents, SLA, CSAT, approvals, departments

1.2 Scope

- 33+ database tables** — Users, tickets, SLA, KB, assets, approvals, templates, etc.
- 80+ API endpoints** — RESTful, JWT-protected
- 15+ frontend pages** — Dashboard, tickets, approvals, hierarchy, templates, reports, etc.

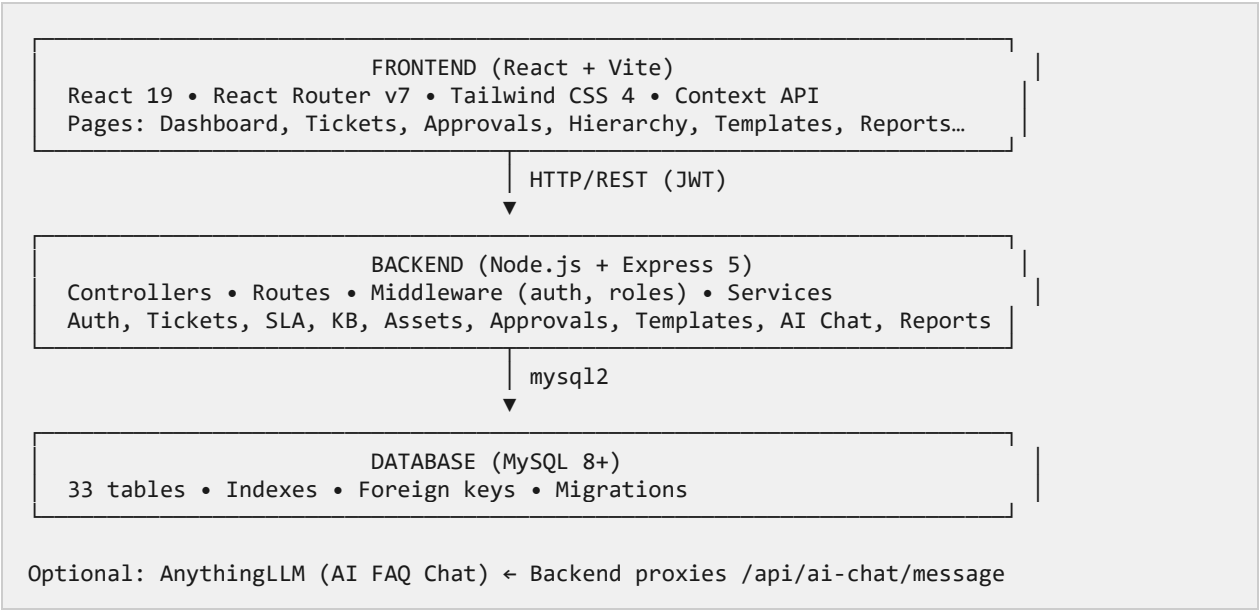
- **150+ documented features** — Ticketing, assets, KB, SLA, automation, notifications, custom fields

1.3 User Roles

Role	Description	Typical Access
Admin	System administrator	Full access, users, SLA, config, reports
Agent	Support staff	Tickets, KB, assets, reports, approvals
Requester	End user / employee	Create tickets, view own tickets, KB

2. System Architecture

2.1 High-Level Architecture



2.2 Backend Structure

```
backend/
├── src/
│   ├── config/           # db.js, migrations, schema SQL
│   ├── controllers/      # auth, ticket, user, team, org, kb, asset, sla,
│   │                       # report, department, hierarchy, approval,
│   │                       # aiChat, template, templateApproval
│   ├── middleware/       # auth.js (JWT, requireAuth, requireRole)
│   ├── routes/           # One file per domain (auth, tickets, users, ...)
│   ├── services/         # approvalWorkflow, approvalSlaService, slaService
│   └── utils/            # roles.js
```

├── seed.js	# Seed users, roles, sample data
├── server.js	# Express app, CORS, route mounting
└── package.json	

2.3 Frontend Structure

frontend/	
├── src/	
│ ├── components/	# AppLayout, FaqChatBar, OrgChart, templates/*,
│ │ ├── tickets/*, ui/* (Button, Card, Modal, ...)	
│ ├── contexts/	# auth, meta, theme
│ ├── pages/	# login, dashboard, tickets, ticketDetail, users,
│ │ ├── teams, hierarchy, approvals, approvalRules,	
│ │ ├── knowledgeBase, assets, sla, reports,	
│ │ └── templateBuilder, profile	
│ ├── services/	# api.js (api(), kbApi, assetsApi, ticketsApi,
│ │ ├── slaApi, reportsApi, tagsApi, auditApi,	
│ │ └── approvalsApi, templatesApi)	
│ ├── styles/	# main.css (Tailwind, theme)
│ ├── App.jsx	# Routes, ProtectedRoute, RoleRoute
│ └── index.jsx	
├── public/	
├── vite.config.js	
└── package.json	

3. Technology Stack

Layer	Technology
Backend	Node.js (ES Modules), Express 5, mysql2, JWT (jsonwebtoken), bcryptjs, express-validator, multer, axios, dotenv, cors
Frontend	React 19, React Router v7, Vite 5, Tailwind CSS 4, Context API, recharts, react-markdown, react-organizational-chart
Database	MySQL 8+ (utf8mb4)
Auth	JWT in Authorization header; optional AnythingLLM API key for AI chat

4. Feature Suite — In-Depth

4.1 Ticket Management

Purpose: Manage support requests through their full lifecycle with status, priority, assignment, and SLA.

Capabilities:

- **Lifecycle:** Create, update, assign, comment, close, reopen. Status flow: New → Open → Pending / On Hold → Solved → Closed.
- **Status transitions (enforced in backend):** new → open, pending, on_hold, solved, closed; open → pending, on_hold, solved, closed; pending/on_hold → open, on_hold/pending, solved, closed; solved → open (reopen), closed; closed → open (reopen only).
- **Classification:** Status, Priority (low, normal, high, urgent), Type (incident, service_request, problem, change), Channel (portal, email, phone, chat, api).
- **Assignment:** Single assignee, primary team; optional multi-team (add/remove teams, complete/reopen team work).
- **Collaboration:** Public/private comments, audit trail (field-level old → new in events).
- **Attachments, tags, events:** File attachments, tag add/remove, event log for status/assignment/field changes.
- **Reopening:** Reopen from solved/closed; reopened_count tracked.
- **Templates:** Tickets can be created from templates (categories, gallery, form builder); template approval flows can be attached.

Key APIs: List/filter tickets, get by id, create, update, bulk update, assign-to-me, escalate, reassign; comments, tags, audit, SLA; multi-team endpoints.

4.2 SLA Management

Purpose: Define policies and measure response/resolution against due times; support approval SLAs.

Capabilities: Policies (response/resolution minutes, optional priority/team, default); ticket SLAs (auto-assignment, response_due_at, resolve_due_at, breach flags); actions (pause, resume, extend, reassign); monitoring (at-risk, breach check, stats); business hours; approval SLAs.

Key APIs: CRUD policies, ticket-SLA get/history, pause/resume/extend/reassign, at-risk, check-breaches, stats, business-hours CRUD, approval-SLA endpoints.

4.3 Approval System (Hierarchical)

Purpose: Require one or more approvers before work is assigned to a team; support rules, departments, and reporting hierarchy.

Capabilities: Approval rules (admin): conditions (priority, type, department, cost); requires_approval, approval levels; workflow on ticket create/send-for-approval; approval chain from user hierarchy; ticket approvals (approve/reject with comments); UI: pending approvals, ticket approval history, approvers list.

Key APIs: Approval rules CRUD, pending approvals, ticket approvals, approve/reject, send-for-approval, approvers.

4.4 Template System

Purpose: Reusable ticket templates with custom form fields and optional multi-step approval flows.

Capabilities: Categories; templates (fields_schema: text, textarea, richtext, select, number, date, file_upload, user_lookup, section_header, etc.); gallery; ticket creation from template; template approval flow per template; ticket template response for display.

Key APIs: Categories CRUD; templates CRUD, duplicate, gallery; ticket template-response; template approval-flow get/save/delete/test.

4.5 Asset Management

Purpose: Track IT assets (hardware/software) and link them to tickets.

Capabilities: Asset types (Computer, Mobile Device, Monitor, Printer, Network Equipment, Software License, custom); assets (asset_tag, name, type, serial_number, manufacturer, model, status, assigned_to_user/org, location, purchase/warranty, notes); link asset to ticket; list tickets for asset.

Key APIs: Asset types CRUD; assets CRUD; link-ticket; get asset tickets.

4.6 Knowledge Base

Purpose: Publish and search articles for self-service support.

Capabilities: Categories; articles (title, body, status draft/published, author, published_at); full-text search; filter by category/status.

Key APIs: Categories CRUD; articles CRUD; search.

4.7 AI FAQ Chat

Purpose: Answer user questions about the Service Desk using a RAG-based AI (AnythingLLM).

Capabilities: Backend proxies to AnythingLLM workspace; FaqChatBar (floating button + expandable chat); conversation history; knowledge source with guardrails.

Key APIs: POST /api/ai-chat/message, POST /api/ai-chat/clear (auth required).

4.8 Organizations & Teams

Purpose: Model companies and support teams for assignment and reporting.

Capabilities: Organizations (name, domain, industry, size, website, notes; members); Teams (name, description; members with is_lead; optional department_id); tickets linked to organization and team(s); multi-team support.

Key APIs: Organizations CRUD; teams CRUD, get members, add/remove members.

4.9 Departments & Hierarchy

Purpose: Organizational hierarchy and reporting chain for approvals and org charts.

Capabilities: Departments (name, description, parent_department_id, head_user_id); user hierarchy (user_id, manager_id, level); org chart API; set/remove manager (admin).

Key APIs: Departments CRUD, get hierarchy; hierarchy: getUserChain, getDirectReports, orgChart, setManager, removeUser.

4.10 Users & Authentication

Purpose: Identify users and enforce role-based access.

Capabilities: Users (email, password_hash, full_name, title, phone, timezone, locale, is_active, last_login_at); roles (admin, agent, requester); Register, login (JWT); GET /auth/me; user management (list, get, create, update, delete; import from Excel).

Key APIs: POST auth/register, POST auth/login, GET auth/me; users CRUD, import-template, import (file upload).

4.11 Reports & Analytics

Purpose: Predefined reports for operations and performance.

Capabilities: Ticket metrics; agent performance; SLA compliance; customer satisfaction (CSAT); ticket trends; team performance; department breakdown; approval metrics; asset summary; resolution distribution; requester activity; hourly heatmap; agent workload; at-risk tickets; SLA by priority; export.

Key APIs: GET /reports/ticket-metrics, agent-performance, sla-compliance, customer-satisfaction, ticket-trends, team-performance, department-breakdown, approval-metrics, asset-summary, resolution-distribution, requester-activity, hourly-heatmap, agent-workload, at-risk-tickets, sla-priority-breakdown, export.

4.12 Dashboard

Purpose: Single-page summary for the logged-in user.

Key API: GET /dashboard (requires auth).

4.13 Meta (Lookups)

Purpose: Provide dropdown/lookup data for statuses, priorities, types, channels.

Key API: GET /meta.

5. Processes & Workflows

5.1 Ticket Lifecycle Process

1. **Create:** Requester or agent creates ticket (optional template). Subject, description, priority, type, channel, requester, optional assignee/team/org. Ticket number generated (e.g. SD-YYYYMMDD-NNNNN). Status = new.
2. **SLA assignment:** Backend assigns SLA policy by priority (and team if configured); sets response_due_at, resolve_due_at in ticket_slas.

3. **Approval (if required):** If an approval rule matches, ticket may go to "pending approval"; approval chain created from hierarchy; first approver notified.
4. **Approval steps:** Each approver approves or rejects with optional comments. If rejected, ticket approval_status = rejected. If all levels approve, approval_status = approved; ticket can then be assigned to team.
5. **Work:** Agent/team updates status, adds comments, assigns, adds tags, links assets. First public comment can mark "response SLA met."
6. **Resolution:** Agent sets status to solved; backend can set closed_at when status is solved/closed. Requester can reopen from solved (reopened_count incremented).
7. **Closure:** Status closed; closed_at set. Reopen only from closed → open.
8. **Audit:** Every status/assignment/field change recorded in ticket_events with old/new values where applicable.

5.2 Approval Workflow Process

1. Rule evaluation on ticket create (or send-for-approval).
2. Chain build from requester's user_hierarchy (N managers); create ticket_approvals.
3. Ticket approval_status = pending.
4. Approver approves or rejects with comments.
5. If approved and more levels exist, next approver notified. If all approved, ticket moves to team queue.
6. Optional approval-SLA policies track time-to-approve.

5.3 Template-Based Ticket Creation Process

1. User selects template from gallery (by category/search).
2. Form rendered from template fields_schema; user fills and submits.
3. Backend creates ticket and stores template response.
4. Template approval flow (if configured) may create approval steps.
5. Ticket detail can show "Template response" tab.

5.4 SLA Breach Detection Process

1. Policies define response_minutes, resolve_minutes.
2. Due dates set at create or policy change.
3. Cron or manual "check breaches" job sets response_breached / resolve_breached.
4. At-risk list shows tickets approaching due.

5.5 AI Chat Process

1. User opens FaqChatBar and types a question.
 2. Frontend sends POST /api/ai-chat/message with message and conversationHistory (JWT).
 3. Backend forwards to AnythingLLM; RAG over uploaded docs.
 4. Backend returns AI reply; frontend appends to conversation.
-

6. API Reference

Base URL: `/api`. All endpoints except auth/register/login and health require `Authorization: Bearer <JWT>`.

6.1 Authentication

Method	Endpoint	Description
POST	<code>/auth/register</code>	Register new user
POST	<code>/auth/login</code>	Login (returns JWT)
GET	<code>/auth/me</code>	Current user + roles

6.2 Tickets

Method	Endpoint	Description
GET	<code>/tickets</code>	List (filter: status, priority, assignee, teamId, organizationId, search, page, pageSize)
GET	<code>/tickets/:id</code>	Get one
POST	<code>/tickets</code>	Create
PATCH	<code>/tickets/:id</code>	Update
PATCH	<code>/tickets/bulk</code>	Bulk update
POST	<code>/tickets/:id/assign</code>	Assign to current user
POST	<code>/tickets/:id/escalate</code>	Escalate priority
POST	<code>/tickets/:id/reassign</code>	Reassign (assignee/team)
GET	<code>/tickets/:id/teams</code>	List teams
POST	<code>/tickets/:id/teams</code>	Add team

PATCH	/tickets/:id/teams/:teamId	Update team
DELETE	/tickets/:id/teams/:teamId	Remove team
POST	/tickets/:id/teams/:teamId/complete	Complete team work
POST	/tickets/:id/teams/:teamId/reopen	Reopen team work
GET	/tickets/:id/sla	Ticket SLA
GET	/tickets/:id/comments	List comments
POST	/tickets/:id/comments	Add comment
GET	/tickets/:id/audit	Audit trail
GET	/tickets/:id/tags	Get tags
POST	/tickets/:id/tags	Add tag
DELETE	/tickets/:id/tags/:tagId	Remove tag
GET	/tags	List all tags

6.3 Approvals

Method	Endpoint	Description
GET	/approval-rules	List rules (admin)
GET	/approval-rules/:id	Get rule
POST	/approval-rules	Create rule
PATCH	/approval-rules/:id	Update rule

DELETE	/approval-rules/:id	Delete rule
GET	/tickets/:id/approvals	Ticket approvals
GET	/approvals/pending	Current user pending
GET	/approvals/history/:ticketId	Approval history
POST	/approvals/:id/approve	Approve
POST	/approvals/:id/reject	Reject
POST	/tickets/:id/send-for-approval	Send for approval
GET	/approvers	List approvers (admin/agent)

6.4 SLA

Method	Endpoint	Description
GET	/sla/policies	List policies (optional ?type=team approval)
GET	/sla/policies/:id	Get policy
POST	/sla/policies	Create (admin)
PUT	/sla/policies/:id	Update (admin)
DELETE	/sla/policies/:id	Delete (admin)
GET	/sla/ticket-slas	List ticket SLAs
GET	/sla/tickets/:ticketId	Ticket SLA
GET	/sla/tickets/:ticketId/history	SLA history

POST	/sla/tickets/:ticketId/pause	Pause
POST	/sla/tickets/:ticketId/resume	Resume
POST	/sla/tickets/:ticketId/extend	Extend
POST	/sla/tickets/:ticketId/reassign	Reassign
GET	/sla/at-risk	At-risk tickets
POST	/sla/check-breaches	Run breach check
GET	/sla/stats	SLA stats
GET	/sla/stats/by-policy	Stats by policy
GET/POST/PUT/DELETE	/sla/business-hours	Business hours CRUD
GET	/sla/approval-slas	Approval SLA list
GET	/sla/approval-slas/stats	Approval SLA stats
GET	/sla/approval-slas/tickets/:ticketId	Ticket approval SLAs
POST	/sla/approval-slas/check-breaches	Approval breach check
GET	/sla/approval-rules	Rules for SLA

6.5 Templates

Method	Endpoint	Description
GET	/templates/categories	List categories
POST	/templates/categories	Create category (admin)

PUT	/templates/categories/:id	Update category
DELETE	/templates/categories/:id	Delete category
GET	/templates/gallery	Gallery (for new ticket)
GET	/templates	List templates
GET	/templates/:id	Get template
POST	/templates	Create (admin)
PUT	/templates/:id	Update (admin)
DELETE	/templates/:id	Delete (admin)
POST	/templates/:id/duplicate	Duplicate (admin)
GET	/tickets/:id/template-response	Template response for ticket
GET	/templates/:id/approval-flow	Get flow (admin)
PUT	/templates/:id/approval-flow	Save flow (admin)
DELETE	/templates/:id/approval-flow	Delete flow (admin)
POST	/templates/:id/approval-flow/test	Test flow (admin)

6.6 Knowledge Base

Method	Endpoint	Description
GET	/kb/categories	List categories
POST	/kb/categories	Create

PUT	/kb/categories/:id	Update
DELETE	/kb/categories/:id	Delete
GET	/kb/articles	List (query params)
GET	/kb/articles/search	Search (?q=)
GET	/kb/articles/:id	Get article
POST	/kb/articles	Create
PUT	/kb/articles/:id	Update
DELETE	/kb/articles/:id	Delete

6.7 Assets

Method	Endpoint	Description
GET	/assets/types	List asset types
POST	/assets/types	Create type
GET	/assets	List assets
GET	/assets/:id	Get asset
POST	/assets	Create
PUT	/assets/:id	Update
DELETE	/assets/:id	Delete
POST	/assets/link-ticket	Link asset to ticket

GET	/assets/:id/tickets	Tickets for asset
-----	---------------------	-------------------

6.8 Organizations & Teams

Method	Endpoint	Description
GET	/organizations	List
POST	/organizations	Create
PATCH	/organizations/:id	Update
DELETE	/organizations/:id	Remove
GET	/teams	List
GET	/teams/:id/members	Members
POST	/teams	Create
PATCH	/teams/:id	Update (admin)
DELETE	/teams/:id	Remove (admin)
POST	/teams/members	Add member (admin)
DELETE	/teams/members/:userId	Remove member (admin)

6.9 Departments & Hierarchy

Method	Endpoint	Description
GET	/departments	List
GET	/departments/:id	Get one

GET	/departments/:id/hierarchy	Hierarchy
POST	/departments	Create (admin)
PATCH	/departments/:id	Update (admin)
DELETE	/departments/:id	Delete (admin)
GET	/hierarchy/user/:id	User reporting chain
GET	/hierarchy/manager/:id/reports	Direct reports
GET	/hierarchy/org-chart	Org chart (admin/agent)
POST	/hierarchy/user/:id	Set manager (admin)
DELETE	/hierarchy/user/:id	Remove from hierarchy (admin)

6.10 Users

Method	Endpoint	Description
GET	/users	List (admin/agent)
GET	/users/import-template	Download import template (admin)
POST	/users/import	Import users Excel (admin)
GET	/users/:id	Get (admin/agent)
POST	/users	Create (admin)
PATCH	/users/:id	Update (admin)
DELETE	/users/:id	Delete (admin)

6.11 Reports

Method	Endpoint	Description
GET	/reports/ticket-metrics	Ticket metrics
GET	/reports/agent-performance	Agent performance
GET	/reports/sla-compliance	SLA compliance
GET	/reports/customer-satisfaction	CSAT
GET	/reports/ticket-trends	Ticket trends
GET	/reports/team-performance	Team performance
GET	/reports/department-breakdown	Department breakdown
GET	/reports/approval-metrics	Approval metrics
GET	/reports/asset-summary	Asset summary
GET	/reports/resolution-distribution	Resolution distribution
GET	/reports/requester-activity	Requester activity
GET	/reports/hourly-heatmap	Hourly heatmap
GET	/reports/agent-workload	Agent workload
GET	/reports/at-risk-tickets	At-risk tickets
GET	/reports/sla-priority-breakdown	SLA by priority
GET	/reports/export	Export (admin/agent)

6.12 Dashboard & Meta

Method	Endpoint	Description
GET	/dashboard	Dashboard summary
GET	/meta	Statuses, priorities, types, channels

6.13 AI Chat

Method	Endpoint	Description
POST	/ai-chat/message	Send message (body: message, conversationHistory)
POST	/ai-chat/clear	Clear conversation (client-side)

6.14 Health

Method	Endpoint	Description
GET	/health	Health check (no auth)

7. Database Schema Summary

- **Auth & users:** roles, users, user_roles
- **Orgs & teams:** organizations, organization_members, teams, team_members
- **Tickets:** ticket_statuses, ticket_priorities, ticket_types, ticket_channels, tickets, ticket_comments, ticket_tags, ticket_tag_links, ticket_attachments, ticket_events
- **SLA:** sla_policies, ticket_slas
- **KB:** kb_categories, kb_articles
- **Assets:** asset_types, assets, asset_ticket_links
- **Satisfaction:** satisfaction_ratings
- **Approvals:** departments, user_hierarchy, approval_rules, ticket_approvals (and related)
- **Templates:** ticket_template_categories, ticket_templates, template approval tables (see migrations)
- **Automation:** automation_rules, automation_logs
- **Email:** email_accounts

- **Business hours:** business_hours, business_hours_schedules
- **Other:** canned_responses, time_entries, notification_preferences, notifications, custom_field_definitions, custom_field_values, saved_filters

Total: 33+ tables. See DATABASE_SCHEMA.md for full column definitions and indexes.

8. Frontend Structure & Routes

8.1 Routes (React Router)

Path	Component	Access
/login	Login	Public
/dashboard	Dashboard	Protected
/tickets	Tickets	Protected
/tickets/new	Redirect to /tickets?create=1	Protected
/tickets/:id	TicketDetail	Protected
/users	Users	Protected, admin/agent
/teams	Teams	Protected
/hierarchy	Hierarchy	Protected, admin/agent
/approvals	Approvals	Protected
/approval-rules	ApprovalRules	Protected, admin
/knowledge-base	KnowledgeBase	Protected
/assets	Assets	Protected, admin/agent
/sla	SlaManagement	Protected, admin

/reports	Reports	Protected, admin/agent
/templates	TemplateBuilder	Protected, admin
/profile	Profile	Protected
*	Redirect to /dashboard	—

8.2 Key Components

- **AppLayout:** Sidebar, header, breadcrumbs, FaqChatBar; wraps all authenticated pages.
- **FaqChatBar:** Floating AI chat (AnythingLLM-backed).
- **OrgChart:** Hierarchy visualization (react-organizational-chart).
- **templates/*:** TemplateBuilder (categories, gallery, form builder, approval flow).
- **tickets/*:** TicketCreateModal, ticket list, ticket detail (conversation, audit, SLA, tags, teams).
- **ui/*:** Button, Card, Badge, Input, Select, Textarea, Modal, Icon, PageHeader.

9. Security & Access Control

- **Authentication:** JWT issued on login; sent as Authorization: Bearer <token>.
- **Authorization:** Middleware requireAuth (valid token), requireRole('admin'|'agent') where needed; requester sees only own tickets.
- **Passwords:** bcrypt hashing; no plaintext storage.
- **SQL:** Parameterized queries (mysql2); no raw concatenation.
- **CORS:** Configured origin (e.g. CORS_ORIGIN env); credentials allowed.
- **Sensitive config:** JWT_SECRET, DB credentials, optional ANYTHINGLLM_URL/API key in env; not committed.

See TEST_CREDENTIALS.md for test accounts and feature access matrix.

10. Setup & Deployment

- **Database:** Create MySQL DB; run migrations from backend/src/config/ (e.g. migrate.js, or run complete-schema.sql); optionally seed (seed.js).
- **Backend:** cd backend, npm install, create .env (PORT, DATABASE_*, JWT_SECRET, CORS_ORIGIN, optional ANYTHINGLLM_URL), npm run dev or npm start.

- **Frontend:** cd frontend, npm install, set VITE_API_URL if needed, npm run dev or npm run build + serve.
- **AI Chat:** Install/run AnythingLLM; create workspace; upload AI_FAQ_KNOWLEDGE_BASE.md; set ANYTHINGLLM_URL and API key.

See SETUP.md for step-by-step and production checklist.

11. Related Documentation

Document	Purpose
FEATURES.md	High-level feature list (150+ items)
DATABASE_SCHEMA.md	Full table definitions and indexes
SETUP.md	Installation, env, API list, default credentials
PLAN.md	Lifecycle & UI revamp plan (Vodafone design)
HIERARCHY_APPROVAL_IMPLEMENTATION_PLAN.md	Approval system design and schema
AI_CHAT_INTEGRATION.md	AI FAQ chat setup and API
AI_FAQ_KNOWLEDGE_BASE.md	RAG content and guardrails for AI
TEST_CREDENTIALS.md	Test accounts and permissions

End of Project Documentation